PROJECT PRESENTATION

# Cricket Player Performance Predictor

Predicting a batter’s expected runs in the next IPL match

[Salman Ahmed-2520089070  P Venkata Harshvardhan-2520089071]  Guide: [Mrs Lakshmi Haritha]
[Department-AIDS] · [KLH university] · [Academic year-2026-27]

# Introduction

### Background

Cricket generates rich ball-by-ball data; IPL matches from 2008 to 2023 are publicly available.

### Domain

Sports analytics and machine learning: turning raw deliveries into player-level insight.

### Why it is needed

Analysts and fans need data-backed expectations of how a batter will perform next.

# Problem Statement

### Existing problem

Expected runs for a batter in the next match are judged by intuition or headline averages.

### Limitations

T20 single-innings scores are highly variable, and raw ball-by-ball data is not player-level.

### Who is affected

Analysts, team management, fantasy-league players and fans.

# Motivation

### Why this project

Combines interest in cricket with an end-to-end machine learning workflow.

### Real-world need

Fantasy leagues, player scouting and match previews all rely on performance expectations.

### Research motivation

Single-match sports prediction is a known hard problem, which makes honest evaluation valuable.

# Objectives

**Main objective:** predict a batter’s expected runs in their next IPL match from historical data.

- Aggregate ball-by-ball deliveries into per-match batting and bowling statistics
- Engineer leakage-safe features from prior matches only
- Train and evaluate a Gradient Boosting Regressor
- Serve predictions through a Flask web interface

# Existing System

### Current approach

Career averages and strike rates read from scorecards, plus manual judgement.

### Technologies

Statistics websites and spreadsheets; no personalised prediction per upcoming match.

### Limitations

Ignores recent form, opponent and venue; not interactive.

# Proposed System

A machine learning model plus a web app that predicts expected runs for a selected batter.

### Recent form

Last-5-innings features capture current form.

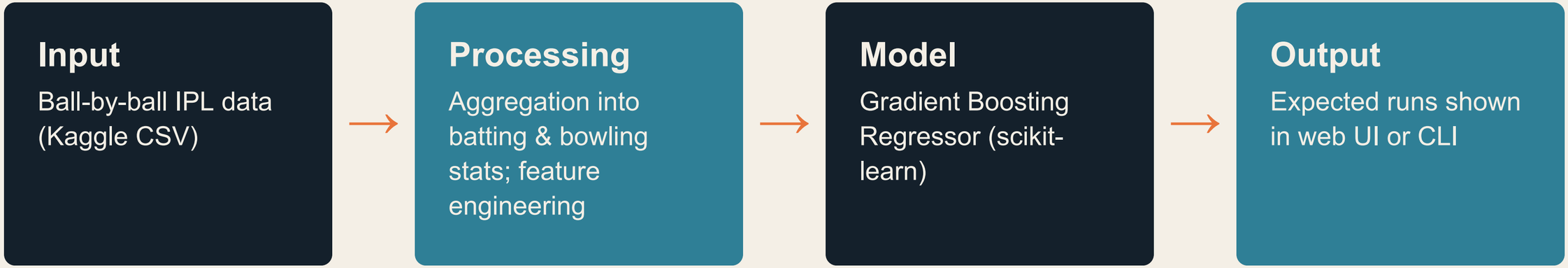
### Context

Opponent and venue averages add match context.

### No leakage

Only earlier matches feed each feature.

# System Architecture



# Methodology

**1 Aggregate**

Group deliveries by match and player: runs, strike rate, boundaries, wickets, economy

→

**2 Engineer**

Career averages, recent form, opponent and venue stats; shift(1) prevents leakage

→

**3 Train**

Gradient Boosting Regressor with a chronological train/test split

→

**4 Evaluate**

MAE and RMSE on later matches

# Technology Stack

| Layer | Tools |
| --- | --- |
| Language | Python 3.14 |
| Data & ML | pandas, scikit-learn (GradientBoostingRegressor), joblib |
| Backend | Flask |
| Frontend | HTML, CSS, vanilla JavaScript |
| Data store & hosting | CSV files, runs locally; Git and GitHub for version control |

# System Modules

### Data pipeline

build_features.py aggregates deliveries

### Feature engineering

Career, recent-form, opponent and venue features

### ML module

Trains and saves model with joblib

### Prediction

predict.py loads model; main.py CLI

### Web UI

server.py with templates and static files

### Storage & security

CSV outputs; no database or login

# Implementation

- Download the Kaggle dataset into dataset/
- Run **python src/build_features.py** to build features and train the model
- Model saved as outputs/player_performance_model.pkl
- Run **python server.py** and open http://127.0.0.1:5000
- Or run **python main.py** for command-line predictions

# Dataset

## Name

Ball-by-Ball IPL (Kaggle: jamiewelsh2/ball-by-ball-ipl), 2008–2023

## Records

[number of deliveries / player-match rows]

## Features

career_avg_runs, career strike rate, last-5 form, opponent and venue averages

## Preprocessing

Per-match aggregation, shift(1) before rolling and expanding calculations

# Algorithm / Model

**Algorithm**

Gradient Boosting Regressor (scikit-learn)

**Why selected**

Handles non-linear feature interactions on tabular data

**Architecture**

Ensemble of sequential decision trees, each correcting earlier errors

**Parameters**

[n_estimators, learning_rate, max_depth]

# Results

| Metric | Baseline (career avg only) | Recent form + opponent/venue |
|---|---|---|
| MAE | 15.52 runs | 15.48 runs |
| RMSE | 20.71 runs | 20.67 runs |

Contextual features gave only a marginal gain, which points to high inherent variance in single-innings T20 batting.

# Result Analysis

**70–80%**

relative importance of career_avg_runs, the dominant feature

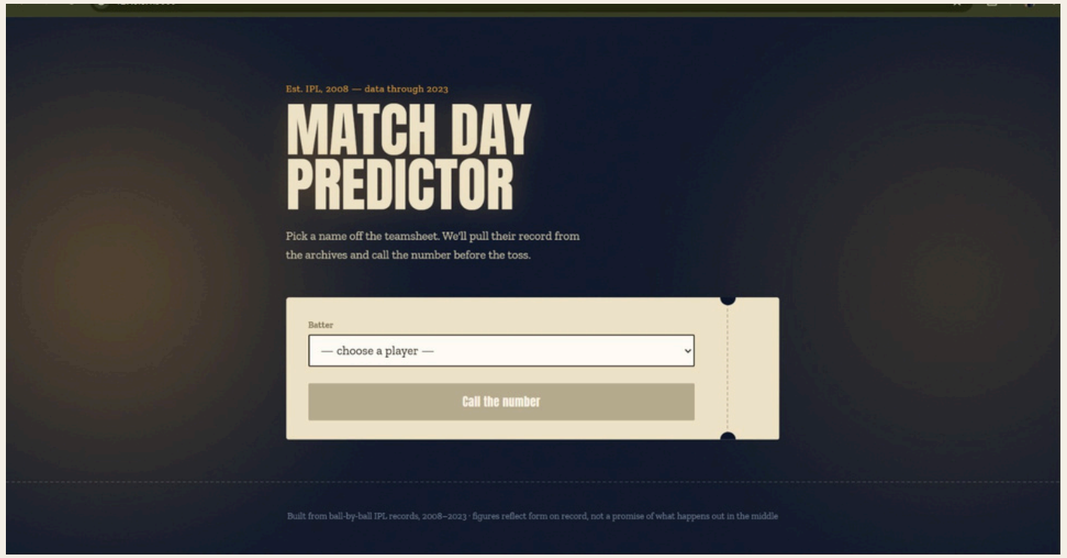
**0.04**

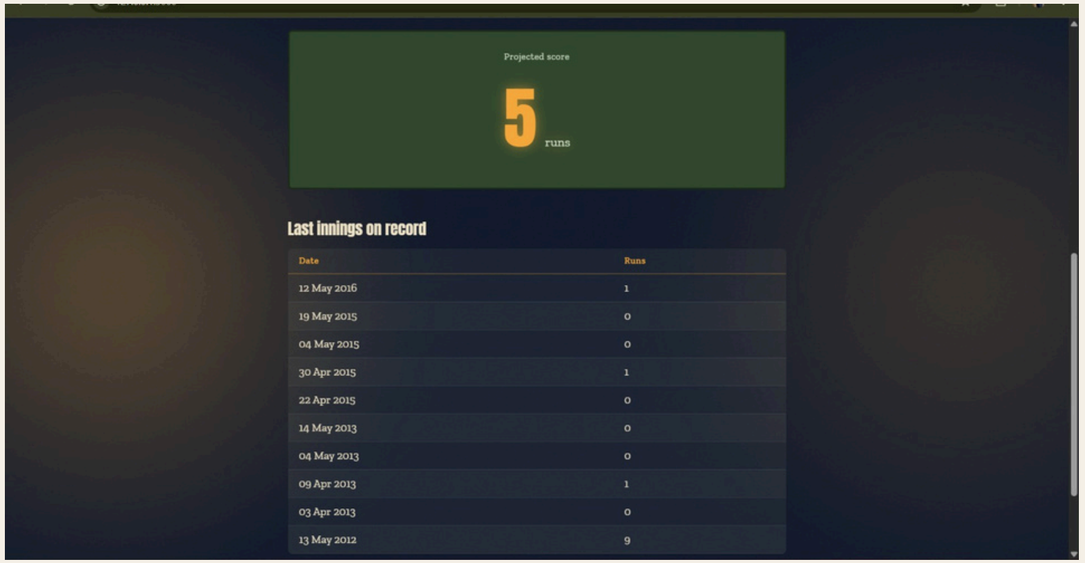
MAE improvement over the baseline, in runs

**N/A**

Accuracy, precision, recall and F1 apply to classification; this is a regression task, so MAE and RMSE are used

# Screenshots / Demo




```
PS C:\player-prediction-model> python src\evaluate.py
=== Regression Metrics (test set) ===
MAE : 15.48 runs
MSE : 427.20
RMSE: 20.67 runs
R²  : 0.111

=== Classification Metrics (above vs. below career average) ===
Accuracy : 0.499
Precision: 0.431
Recall   : 0.633
F1 score : 0.513

Confusion matrix:
[[ 682 1010]
 [ 443  764]]
PS C:\player-prediction-model>
```

# Advantages

- Leakage-safe features give an honest, realistic evaluation
- Chronological split mirrors real forecasting
- Interactive web interface and CLI
- Adds recent form and opponent/venue context beyond career averages

# Applications

### Real-world uses

Fantasy cricket, match previews, player scouting

### Target users

Analysts, coaches, broadcasters, fans

### Industries

Sports analytics, media, fantasy gaming

# Limitations

### Current

Only marginal gain over the baseline; predicts batting runs only

### Data

Covers IPL 2008–2023; single-innings variance is high

### Technical

Runs locally; no bowler-vs-batter or match-situation context

# Future Scope

- Predict over a rolling next-3-match window, or classify above/below average
- Add bowler-vs-batter matchup history
- Add powerplay/death-overs role and batting position
- Extend to bowling performance; deploy online

# Conclusion

### Achieved

End-to-end pipeline: data, features, model and web app

### Key outcome

MAE 15.48 and RMSE 20.67 runs; career average dominates

### Contribution

A leakage-safe baseline and honest evidence of how hard single-match prediction is